

Kompleksowy Raport Badawczy: Radykalna Optymalizacja i Przeprojektowanie Architektury Asynchronicznego Potoku Webhooków w Ekosystemie AWS

Wprowadzenie i Ocena Stanu Obecnego

We współczesnych rozproszonych systemach informatycznych, opartych na architekturze sterowanej zdarzeniami (Event-Driven Architecture), sprawna i niezawodna obsługa webhooków stanowi jeden z najważniejszych filarów stabilności operacyjnej. Analiza przedstawionej architektury referencyjnej, w której zewnętrzny dostawca przesyła zdarzenia poprzez żądania HTTP POST do usługi Amazon API Gateway, skąd ruch jest kierowany do warstwy Ingress zbudowanej w oparciu o framework NestJS oraz środowisko AWS Lambda, ukazuje klasyczne i powszechnie stosowane podejście. W tym modelu instancja przyjmująca żądanie weryfikuje surowy ładunek kryptograficzny (Raw Body Middleware) w celu uwierzytelnienia podpisu, a następnie – po pomyślnej autoryzacji – natychmiast zwraca kod HTTP 202 Accepted, odkładając równocześnie wiadomość na kolejkę Amazon SQS. Z kolejki tej zdarzenia są pobierane przez asynchroniczne procesy robocze (Worker Consumer), również oparte na frameworku NestJS, które weryfikują idempotencyjność operacji przed ostatecznym zapisem stanu do bazy danych.

Choć powyższy model wydaje się poprawny koncepcyjnie, dogłębna ocena inżynierska ujawnia szereg krytycznych braków, ukrytych barier wydajnościowych oraz ograniczeń architektonicznych, które dyskwalifikują to rozwiązanie w kontekście ekstremalnego obciążenia produkcyjnego (High-Concurrency). Rozwiązanie to jest podatne na degradację wydajności wynikającą z tzw. zimnych startów (cold starts) środowiska uruchomieniowego, fizyczne ograniczenia rozmiaru przesyłanych wiadomości, a także skomplikowane i trudne do debugowania problemy z wyczerpywaniem się puli połączeń do relacyjnych baz danych. W warunkach produkcyjnych, gdzie systemy muszą radzić sobie z nieprzewidywalnymi falami ruchu (tzw. traffic spikes), przestarzałe podejście polegające na angażowaniu "ciężkich" kontenerów inwersji sterowania (DI) jedynie do walidacji nagłówków kryptograficznych generuje nieuzasadnione koszty i stwarza ryzyko kaskadowych awarii.

Niniejszy raport stanowi wyczerpujące kompendium analityczne, w którym zidentyfikowano i przeanalizowano wszystkie słabe punkty obecnego systemu. Zawiera on propozycję przełomowego, futurystycznego rozwiązania, które radykalnie przeprojektuje cały przepływ danych, przesuwając ciężar weryfikacji na krawędź sieci (Edge Computing) i wprowadzając w pełni zarządzane usługi do orkiestracji strumieni. W ramach dokumentu zaprezentowano wizualizacje struktur, szczegółowy plan migracji oraz kompletny, gotowy do wdrożenia na produkcji ("turbo szczegółowy") plan operacyjny dla zrekonstruowanego ekosystemu.

Dekonstrukcja Ukrytych Barrier i Ograniczeń Wydajnościowych

Analiza obecnej architektury pozwala na wyodrębnienie kluczowych elementów, które tworzą ukryte i bezpośrednie bariery dla maksymalizacji wydajności, elastyczności oraz satysfakcji użytkowników. Zrozumienie mechanizmów powstawania tych barier jest niezbędne do zaprojektowania docelowego, przełomowego systemu.

Paradoks Autoryzacji i Wąskie Gardło Warstwy Ingress

Najbardziej fundamentalnym problemem zaprezentowanego przepływu jest umieszczenie weryfikacji podpisu w warstwie aplikacji (Ingress Layer) opartej na NestJS. Framework ten, z uwagi na swoją architekturę zorientowaną na modułowość i zaawansowane wstrzykiwanie zależności, wymaga znacznego czasu na inicjalizację. W architekturze Serverless, gdzie instancje AWS Lambda są dynamicznie powoływane do życia, prowadzi to do zjawiska zimnych startów, które mogą trwać od kilkuset milisekund do kilku sekund.

Dostawcy webhooków, tacy jak systemy płatności czy platformy e-commerce, implementują rygorystyczne zasady określające dopuszczalny czas oczekiwania na odpowiedź ze strony klienta (często poniżej kilku sekund). Jeśli instancja NestJS nie zdąży się uruchomić, poprawnie zinterpretować surowego ciała żądania (Raw Body) i wygenerować kodu HTTP 202, dostawca uzna próbę za nieudaną i wdroży mechanizm ponowień (retries). Prowadzi to do efektu kuli śnieżnej: system jest zalewany zduplikowanymi żądaniami, podczas gdy kolejne instancje Lambda próbują się zainicjalizować.

Co więcej, próba przeniesienia logiki autoryzacji do natywnych rozwiązań, takich jak AWS API Gateway Custom Lambda Authorizer, natrafia na architektoniczną przeszkodę. Autoryzatory Lambda w usłudze API Gateway otrzymują w obiekcie zdarzenia (event) nagłówki, ścieżki i parametry zapytania, ale celowo nie mają dostępu do ciała żądania (request body). Jest to spowodowane optymalizacją przepustowości i limitami rozmiaru danych, jednak uniemożliwia kryptograficzną weryfikację podpisu HMAC, która wymaga przeliczenia skrótu na podstawie pełnego, niezmienionego ładunku (Raw Body) i porównania go z wartością z nagłówka. Z tego powodu deweloperzy są zmuszeni do przepuszczania ruchu w głąb systemu i obciążania warstwy aplikacyjnej logiką, która powinna być zrealizowana w strefie DMZ. Wymaga to kosztownego i powolnego odzyskiwania surowego bufora (Buffer) na poziomie żądania frameworka Express, zanim nastąpi deserializacja JSON.

Ograniczenia Rozmiaru Ładunku (Payload Size Limits) i Ryzyko Utraty Danych

Kolejnym poważnym brakiem w obecnym systemie jest bezpośrednie mapowanie danych z API Gateway do Amazon SQS. Kolejki SQS w środowisku AWS posiadają fizyczne, bezwzględne ograniczenie rozmiaru pojedynczej wiadomości, wynoszące 256 KB. Choć AWS zwiększył niedawno maksymalny rozmiar ładunku dla asynchronicznych wywołań samej funkcji Lambda do 1 MB, system przesyłania wiadomości (broker) w postaci SQS pozostaje wąskim gardłem. Dostawcy tacy jak Stripe czy platformy klasy ERP mogą emitować webhooks zawierające obszerne metadane, zagnieżdżone struktury transakcyjne lub wręcz rzuty obiektów, których rozmiar z łatwością przekracza bezpieczną barierę kilkudziesięciu kilobajtów. Próba

bezpośredniego zapisu tak dużej wiadomości na kolejkę zakończy się twardym odrzuceniem (HTTP 413 Payload Too Large lub wewnętrznym błędem integracji). Obecna architektura nie przewiduje mechanizmu łagodzenia tego problemu, co prowadzi do bezpowrotnej utraty kluczowych zdarzeń biznesowych. Zjawisko to obniża jakość danych oraz zagraża spójności operacyjnej między systemem wewnętrznym a zewnętrzną dostawcą.

Wyczerpanie Puli Połączeń Relacyjnej Bazy Danych i Zjawisko Connection Pinning

Przetwarzanie dużej liczby komunikatów w modelu rozproszonym wymusza jednocześnie powoływanie setek, a nawet tysięcy instancji środowiska uruchomieniowego. W systemach wykorzystujących relacyjne bazy danych (np. PostgreSQL czy MySQL w ramach usługi Amazon Aurora), każda aktywna instancja asynchronicznego konsumenta SQS otwiera osobne połączenie sieciowe z bazą danych. Szybko doprowadza to do wyczerpania maksymalnej liczby dozwolonych połączeń w klastrze bazodanowym, prowadząc do zatorów, zwiększonych opóźnień operacji wejścia/wyjścia oraz niestabilności.

Standardowym, powszechnie wdrażanym rozwiązaniem jest implementacja usługi Amazon RDS Proxy, która działa jako warstwa pośrednicząca, zarządzając pulą ciepłych połączeń i współdzieląc je pomiędzy wieloma klientami. Architektura wydaje się bezpieczna, jednak ukrywa krytyczną barierę wydajnościową znaną jako "Connection Pinning" (przypinanie połączeń). RDS Proxy optymalizuje zapytania poprzez ich przeplot, ale ucieka się do przypinania konkretnej fizycznej sesji bazy danych do pojedynczego połączenia klienckiego w momencie, gdy rozmiar tekstu pojedynczego zapytania SQL przekroczy 16 KB.

Z uwagi na naturę webhooków, które często oznaczają konieczność zapisu potężnych zrzutów JSON do kolumn typu JSONB w systemach ORM (takich jak TypeORM w środowisku NestJS), zapytania INSERT i UPDATE masowo przekraczają granicę 16 KB. RDS Proxy, broniąc integralności danych przed nadpisaniem zmiennych sesyjnych, wyłącza dla nich mechanizm multipleksowania. Skutkuje to lawinowym przyrostem przypiętych połączeń, ominięciem zalet Proxy i ostatecznie zablokowaniem całego silnika bazy danych pod obciążeniem.

Brak Rozproszonej i Skalowalnej Idempotencji Zabezpieczonej przed Wyścigami

Model rozproszony, w którym broker (AWS SQS) gwarantuje dostarczenie wiadomości "co najmniej raz" (at-least-once delivery), narzuca bezwzględny obowiązek implementacji logiki idempotentnej. Bez niej, ponowne przetworzenie tego samego komunikatu (np. w wyniku błędu sieciowego lub duplikacji w SQS) spowoduje zwielokrotnienie ubocznych efektów biznesowych, takich jak ponowne zaksięgowanie wpłaty.

Wskazana w diagramie warstwa [Idempotency Guard] często bywa błędnie implementowana w warstwie logiki aplikacji poprzez prosty odczyt stanu z bazy danych (SELECT), a następnie, w przypadku braku rekordu, wykonanie logiki i aktualizację (INSERT/UPDATE). Takie podejście tworzy klasyczny problem wyścigu (check-then-act race condition). Jeżeli system przetwarza masowo tysiące komunikatów na sekundę, dwa duplikaty tego samego webhooka mogą trafić do różnych instancji przetwarzających niemal w tym samym ułamku sekundy. Obie instancje odczytają stan początkowy, stwierdzą brak wcześniejszego wykonania zadania i równolegle przystąpią do realizacji funkcji. Prowadzi to do naruszenia spójności danych. Bez zastosowania gwarancji atomowych na poziomie systemu zewnętrznego (np. bazy operacyjnej) architektura

pozostaje wysoce niestabilna.

Nadmierna Złożoność i Kod Schematyczny (Boilerplate) w Obsłudze SQS

W środowisku NestJS implementacja konsumenta SQS wymaga zazwyczaj zastosowania dedykowanych bibliotek i modułów, które ciągle i aktywnie nasłuchują nowych wiadomości (long polling). Rozwiązanie to nakłada na inżynierów obowiązek ręcznego pisania kodu odpowiedzialnego za zarządzanie czasem widoczności (visibility timeout), obsługę mechanizmu powtórek (retries), parsowanie plików JSON oraz interakcje z infrastrukturą AWS. Konieczność umieszczenia w warstwie biznesowej kodu infrastrukturalnego jest w bezpośredniej sprzeczności z filozofią czystej architektury (Clean Architecture) i drastycznie zwiększa koszt utrzymania (dług techniczny).

Alternatywne Ścieżki w Ekosystemie AWS i Ich Ocena

Aby zapewnić najbardziej przemyślany i wyczerpujący plan, raport ocenia różne mechanizmy i opcje chmurowe w ekosystemie AWS, które mogłyby zastąpić lub wesprzeć analizowaną architekturę. Rola Principal Engineer wymaga dogłębnego przemyślenia, które narzędzia stanowią optymalny punkt równowagi między skalowalnością, kosztami a funkcjonalnością.

Opcja Technologiczna	Charakterystyka W Ekosystemie AWS	Zastosowanie w Analizowanym Przypadku
API Gateway vs. Application Load Balancer (ALB)	API Gateway oferuje wbudowaną walidację, limitowanie zapytań (throttling), klucze API oraz łatwą integrację z usługami AWS bez uruchamiania zasobów obliczeniowych (VTL). ALB wymaga utrzymania aktywnych celów pod spodem (np. Fargate lub pracujących Lambda) w celu przetworzenia jakiegokolwiek żądania.	Zalecane: API Gateway. Brak konieczności płacenia za zasoby spoczynkowe. Ochrona w warstwie L7 oraz ułatwione limitowanie złośliwego ruchu.
Amazon SQS vs. Amazon SNS vs. Amazon EventBridge	SQS to klasyczna kolejka punkt-punkt z gwarancją asynchronicznego dostarczania. SNS służy do rozsyłania wiadomości w architekturze Publikuj/Subskrybuj (Pub/Sub). EventBridge (Event Bus) pozwala na zaawansowany routing, filtrowanie i integrację bezpośrednio z aplikacjami SaaS bez potrzeby pisania kodu webhooków, obsługując	Zalecane: Połączenie SQS i EventBridge Pipes. Event Bus jest świetny do routingu na wiele celów, jednak kosztuje ok. \$1.00 za milion zdarzeń. Z kolei usługa EventBridge Pipes to koszt \$0.40/milion, dedykowany do optymalnego pompowania danych z pojedynczego źródła (SQS) bezpośrednio do pojedynczego celu (Lambda), z możliwością filtrowania i wzbogacania danych po

Opcja Technologiczna	Charakterystyka W Ekosystemie AWS	Zastosowanie w Analizowanym Przypadku
	topologię wielu celów.	drodze.
Obliczenia w Tle: AWS Lambda vs. AWS Fargate (ECS)	Lambda pozwala na skalowanie do zera, płatność wyłącznie za czas wykonania rzędu milisekund. Fargate (ECS) wymaga podniesienia zadania i długiego uruchamiania, ale radzi sobie świetnie ze stabilnym ruchem i aplikacjami stanowymi.	Zalecane: AWS Lambda dla architektury zdarzeniowej sterowanej webhookami, gdzie zmienność i piki ruchu mogą być nagłe. Ogranicza to koszty stałe. W przypadkach ekstremalnego obciążenia i ciągłych połączeń z bazą danych można przenieść konsumpcję do Fargate, ale nowe technologie łagodzą ten wymóg.

Przełomowe Rozwiązanie Futurystyczne: "Zero-Compute Cognitive Edge Fabric"

Mając na uwadze wszystkie zidentyfikowane luki, ograniczenia i alternatywy, poniżej zaproponowano rewolucyjną, odważną technologię będącą uosobieniem koncepcji "idealnej wersji" systemu. Rozwiązanie to wybiega poza dzisiejsze standardy, łącząc natywną chmurę brzegową z inteligentną automatyzacją oraz głęboką kontrolą stanu. System ten nazwany został **"Zero-Compute Cognitive Edge Fabric"** (Bezoserwerowa, Kognitywna Matryca Brzegowa). Istotą tej innowacji jest radykalne wyeliminowanie logiki zarządzającej ruchem i weryfikacją na poziomie warstwy pośredniczącej, a także wprowadzenie sztucznej inteligencji do samodzielnej remediacji błędów.

Filary Przełomowej Koncepcji:

1. **Holograficzna Kryptografia Brzegowa (CloudFront Functions WebCrypto):** W idealnym systemie autoryzacja odbywa się milisekundy od miejsca, z którego przyszło zapytanie użytkownika, zanim dotrze do rdzenia infrastruktury w chmurze. Zamiast obciążać instancje NestJS/Lambda, żądania obsługiwane są przez AWS CloudFront. Wykorzystanie usługi CloudFront Functions z nowym środowiskiem wykonawczym JavaScript Runtime 2.0 zapewnia natywny dostęp do interfejsu WebCrypto API. Moduł crypto w CloudFront Functions może zweryfikować sygnaturę wiadomości w ułamku milisekundy. Jeśli wynik negocjacji w stałym czasie się nie powiedzie, ruch jest dławiony u samego brzegu (Edge), zwracając kod błędu, zanim AWS naliczy jakiegokolwiek opłaty za wejście w głąb infrastruktury API Gateway.
2. **Architektura Transmisyjna Claim-Check (S3 + VTL Direct Integration):** Zamiast przepuszczać ruch do dedykowanego serwisu przyjmującego, API Gateway posłuży tu jako mechanizm translacyjny. W oparciu o Velocity Template Language (VTL), żądanie zostaje natychmiast asynchronicznie przemapowane na komendę SendMessage dla Amazon SQS. Natomiast w obliczu potencjalnego przekroczenia bariery 256 KB, wprowadzany jest zaawansowany wzorzec Claim-Check. W takim wypadku ładunek w całości zapisywany jest bezpośrednio do bufora w Amazon S3. Do kolejki trafia jedynie

wskaźnik (tzw. "Bilet z Odbiorni", np. nazwa bucketu S3 i wygenerowany unikalny klucz obiektu).

3. **Kognitywna Pętla Samonaprawy (AI-Driven Self-Healing na Amazon Bedrock):**
Funkcją budzącą największe zaskoczenie i radykalnie definiującą branżę na nowo jest wprowadzenie pętli sprzężenia zwrotnego na kolejkach martwych listów (Dead Letter Queues - DLQ) z wykorzystaniem modelu dużego języka (LLM). Kiedy zewnętrzny dostawca webhooka nagle i bez uprzedzenia zmieni strukturę JSON (schema drift), konwencjonalne aplikacje przestaną parsować ładunek, wyrzucając błędy i gromadząc stos logów w DLQ, aż inżynier zapozna się z powiadomieniem. W "Kognitywnej Matrycy", pojawienie się nowej kategorii błędów automatycznie uruchamia wyizolowaną funkcję analizatora AWS Lambda z dostępem do usługi Amazon Bedrock. Analizator zbiera i dekoduje surowy ładunek z DLQ, zestawia go ze zrzutem śladu stosu błędów (Stack Trace) oraz z kodem aktualnego schematu (DTO w repozytorium). Generatywna sztuczna inteligencja odnajduje brakujące pole, samodzielnie transformuje kod TypeScript aplikacji i automatycznie tworzy zgłoszenie zmian (Pull Request), połączone z natychmiastowym obejściem problemu w API Gateway, pozwalając na ponowne przetworzenie komunikatu bez przestojów biznesowych.

Wizualizacja Map Architektur i Projekt Przejęcia

Aby unaocznić transformację, poniżej zaprezentowano wizualizacje za pomocą struktury schematów ukazującej stan niedoskonały, mapę braku oraz docelową, idealną architekturę.

Mapa Braku – Stan Niedoskonały (Current State)

```
| ▼ ————— | | ▼ ▼ [Ingress Layer] ◀—▶ (NestJS/Lambda) | | ▼ | | | ▼ ▼
[Immediate 202 ACK] | ▼
| ▼
| ▼ [Idempotency Guard] | ▼
```

Przepływ Docelowy – "Naprawiony" Zoptymalizowany System

```
| (HTTP POST with HMAC Signature) ▼ ===== ZABEZPIECZENIE
BRZEGOWE (EDGE) ===== [Amazon CloudFront] | |—▶ —▶
Weryfikacja WebCrypto — (Błąd: 401) | ▼ (Sukces: < 1ms) =====
NATYCHMIASTOWA INTEGRACJA & BUFOR =====
| |—▶ Wzorzec Claim-Check: Czy Payload > 256KB? | |—▶ TAK —▶ Zapis całego Body
do | | |—▶ Zwraca Wskaźnik (S3 Object URI) | |—▶ NIE —▶ Zachowaj Payload | |—▶
—▶ Przekazuje ładunek lub wskaźnik do SQS | |—▶ Natychmiastowy zwrot HTTP 202
Accepted (Bezkosztowo) ▼
| ===== ORKIESTRACJA & WZBOGACANIE =====
| (Zarządzany Polling, batching i filtrowanie - brak kodu w NestJS) |—▶ Krok Enrich (Opcja):
Jeśli Claim-Check, pobierz pełny obiekt z S3 ▼ ===== WYKONANIE &
ŻELAZNY STAN =====
| |—▶ —▶ | (Gwarantowana atomowość: IN_PROGRESS -> COMPLETED) ▼ ◀—▶
(Mechanizm Proxy Splitting) | ===== KOGNITYWNA SAMONAPRAWA (FUTURYSTYCZNY
ELEMENT) ===== | (Jeśli błąd walidacji schematu ulegnie procesom ponowienia i trafi do
```

DLQ) ▼

| ▼ —► Korelacja Payloadu z kodem źródłowym | L► Wdrożenie szybkiej poprawki i generowanie Auto-Pull Request

Plan Przejścia (Migracja Zoptymalizowanego Systemu)

Przeprowadzenie udanej integracji wszystkich elementów wymaga metodycznego usunięcia każdej z barier, skupiając się na działaniach o największym, najbardziej zauważalnym przełożeniu na stabilność.

Priorytet	Zidentyfikowana Bariera	Przyczyna Braku	Małe, ale Wysoko Wpływowe Działanie Rozwiązujące Problem
1 (P0)	Korupcja danych wywołana wyścigami równoległych duplikatów.	Brak mechanizmu atomowego blokowania stanu operacji we wcześniejszej fazie. Standardowa baza SQL oparta na odczycie/zapisie ulega w środowisku wielowątkowym.	Zainstalowanie paczki AWS Lambda Powertools i dodanie dekoratora @Idempotent wraz z dedykowaną tabelą DynamoDB, wprowadzając rygorystyczne blokowanie warunkowe.
2 (P1)	Wąskie gardło opóźnień Ingress (Cold Starts) uniemożliwiające zwrócenie w czasie kodu HTTP 202.	Framework NestJS uruchamia zaawansowane kontenery inwersji sterowania, aby zrealizować proste żądanie weryfikacji nagłówków.	Wdrożenie skryptu JavaScript (Runtime 2.0) na CloudFront Functions wykorzystującego funkcję crypto.subtle.verify. Wyłącza to całkowicie potrzebę istnienia NestJS w warstwie publicznej strefy zdemilitaryzowanej.
3 (P1)	Odrzucenia ogromnych webhooków oraz destabilizacja warstwy bazy.	SQS nakłada twardy limit wielkości (256KB), a zapytania ORM powyżej 16KB omijają pulę RDS Proxy ze względu na proces <i>Connection Pinning</i> .	Wdrożenie wzorca Claim-Check na S3 dla wszystkich ładunków, a z perspektywy bazy danych skonfigurowanie dwóch osobnych źródeł TypeORM (DataSources), kierując "ciężkie" zapisy na wydodrębniony klaster proxy (Proxy Splitting).

Priorytet	Zidentyfikowana Bariera	Przyczyna Braku	Małe, ale Wysoko Wpływowe Działanie Rozwiązujące Problem
4 (P2)	Zaśmieszenie logiki biznesowej kodem pętli odpytującej.	Wykorzystanie zewnętrznych bibliotek (np. sqs-consumer) do bezpośredniej asynchronicznej obsługi kolejkowania, wprowadzających skomplikowany, podatny na błędy kod infrastrukturalny.	Usunięcie kodu obsługi SQS z NestJS. Konfiguracja Amazon EventBridge Pipes jako zarządzanego pomostu bezpośrednio wywołującego funkcję Lambda po napłynięciu danych.
5 (P3)	Długi czas detekcji zmian na produkcyjnych schematach od dostawców zewnętrznych.	Awaryjność integracji API wynika z niespodziewanych decyzji innych zespołów programistycznych po stronie źródła.	Stworzenie automatyzacji reagującej na zdarzenia w Dead Letter Queue przy pomocy AI, analizującej strukturę błędu LLM-em w poszukiwaniu nowych pól.

Turbo-Szczegółowy Dokument Konfiguracyjny (Production Blueprint)

Część wdrożeniowa poniżej stanowi absolutnie wyczerpujący komplet technicznych i aplikacyjnych kroków potrzebnych do konfiguracji ekosystemu Zero-Compute Cognitive Edge Fabric w reżimie produkcyjnym. Wyznacza ona metody stosowania innowacyjnych rozwiązań bezpośrednio na środowiskach AWS.

Krok 1: Weryfikacja Podpisu Kryptograficznego za Pomocą CloudFront Functions

Warstwa brzegowa odpowiedzialna jest za absolutne odrzucenie jakichkolwiek żądań o niepoprawnej walidacji HMAC z opóźnieniem nieprzekraczającym 1 milisekundy. Ponieważ środowisko wykonawcze Node.js o pełnym spektrum pakietów (crypto z NPM) jest niedostępne na krawędzi bez gigantycznych obciążeń architektonicznych, wykorzystany zostanie wbudowany natywnie w środowisku JavaScript Runtime 2.0 standard WebCrypto API. Na produkcji należy zaimplementować funkcję, która asynchronicznie parsuje nagłówki oraz treść. Klucz udostępniany przez dostawcę webhooka (Webhook Secret) powinien być trzymany w pamięci szybkiej AWS CloudFront KeyValueCollection i powiązany z naszą funkcją.

```
import crypto from 'crypto';
// Stały klucz KVS pozyskany bezpiecznie:
const kvsKey = 'jwt.secret';
```

```
async function handler(event) {
```



```

const request = event.request;
const body = request.body.data;
// Odtwarzanie sygnatury poprzez asynchroniczne funkcje wspierane od wersji JS 2.0
const signature = request.headers['webhook-signature'].value;
// Przeliczenie HMAC-SHA256 za pomocą webowej biblioteki wbudowanej:
// Procedura negocjacji sygnatury kryptograficznej musi opierać się o operacje
// w czasie stałym (constant-time compare) by zapobiec atakom czasowym (Timing Attacks).
}

```

Zastosowanie algorytmu opartego na negocjacji przez bramki XOR po bajtach (lub użycie udostępnionej struktury równego czasu) zapewnia, że atakujący nie odgadnie prawidłowego podpisu poprzez statystyczną analizę mikrosekundowych różnic czasu wykonania. Żądania zweryfikowane pozytywnie przepuszczane są do API Gateway.

Krok 2: Transformacja API Gateway i Wzorzec Claim Check z VTL

Autoryzowany ruch wchodzi do Amazon API Gateway. Skonfigurowane zostaje pojedyncze wejście typu REST (HTTP API jest prostsze, ale REST posiada szersze opcje mapowań) zintegrowane bezpośrednio z Amazon SQS z wykorzystaniem żądania POST Action=SendMessage.

Aby przeciwdziałać błędom związanym z rozmiarem ładunku (powyżej 256 KB) wprowadza się technikę hybrydową. Ruch analizowany jest przez VTL. Jeśli obliczony zostanie nadmiar objętości (lub jeśli jest taka biznesowa potrzeba bezpieczeństwa), mechanizm API Gateway zapisuje plik pełnego zdarzenia do kubka S3, nadając mu losowy identyfikator. W SQS generowana jest tylko referencja – struktura "biletowa" ukształtowana następująco:

Technologia EventBridge Pipes, omawiana w kroku trzecim, odtworzy ten obiekt bezpośrednio do aplikacji. Wyzwała to system do obsługi teoretycznie nielimitowanych rozmiarów zdarzeń wejściowych z jednoczesnym, zjawiskowo tanim rozwiązaniem magazynowania i czyszczenia poprzez Amazon S3 Lifecycle Rules.

Krok 3: Orkiestracja Usługą Amazon EventBridge Pipes

Rozwiązania oparte na pakiecie sqs-consumer czy nestjs-sqs dla frameworka NestJS są kłopotliwe i niestabilne pod obciążeniem, narażając środowiska na niepotrzebne przepełnienia wywołań zwrotnych i problem zarządzania czasem blokady SQS.

Dlatego na poziomie orkiestracji definiujemy Amazon EventBridge Pipes jako natywny łącznik. Zastosowanie tej topologii uwalnia aż do 60% redukcji kosztów operacyjnych w stosunku do stosowania pełnego Amazon EventBridge Event Bus.

- **Source:** Ustawiamy Amazon SQS (lub SQS FIFO, gdzie Message Group ID zachowa kolejność logiczną). Ograniczamy parametry jednoczesnego zapytań (Batch Size) do 10-100 obiektów, celem kontrolowanej propagacji do funkcji.
- **Enrichment:** Opcjonalnie, gdy w pakiecie zidentyfikowany zostanie bilet z Amazon S3 (wzorzec Claim-Check), rura wywołuje na ułamek sekundy skróconą procedurę pobierającą właściwy ładunek w formacie tekstowym, podmieniając go w pule. W rezultacie system biznesowy zawsze pracuje z w pełni skonsolidowanym ciągiem JSON, nie mając w ogóle koncepcji podziału infrastrukturalnego.
- **Target:** Skonfigurowana zostaje instancja AWS Lambda działająca jako samodzielna

usługa na bazie

NestFactory.c[span_67](start_span)[span_67](end_span)reateApplicationContext
(Standalone Application). Minimalizuje to czas startu bez inicjalizowania kosztownego
serwera HTTP warstwy widoku.

Krok 4: Żelazna Gwarancja Operacji (AWS Powertools Idempotency na DynamoDB)

Podwójne dostarczenie tej samej wiadomości to naturalny objaw gwarancji AWS "at-least-once". Próba polegania na "unikalnych restrykcjach tabel SQL" (Unique Constraints) oraz instrukcji blokujących optymistycznie stwarza jednak problemy martwych punktów, wydajności oraz nakłada podatek na bazę z racji ciągłego tworzenia zapytań. Najbardziej zaawansowana praktyka polega na wymuszeniu idempotencji poprzez oddzielny zoptymalizowany składnik chmury – noSQL Amazon DynamoDB – połączony z paczką powertools.

Programiści inicjują strukturę bazy danych o kluczu głównym id i zdefiniowanym trybie płatności (PAY_PER_REQUEST), redukując do minimum koszty jałowe. Logika polega na nałożeniu wyrażenia warunkowego przez dekorator lub Middleware @Idempotent(), który:

1. Rozpatruje ścieżkę parametru zdarzenia wejściowego (np. eventKeyJmesPath: "[\"id\", \"type\"]") jako identyfikator transakcji.
2. Tworzy atomowy insert (PutItem) z instrukcją attribute_not_exists(id) statusu wpisu jako IN_PROGRESS z wyliczonym polem czasu zgonu (TTL – Expiration) w DynamoDB.
3. Jeśli operacja ulegnie błędowi warunkowemu (ConditionalCheckFailedException), powertools przerywa działanie i rzuca krytyczny błąd wskazując zduplikowany postęp, zmuszając EventBridge Pipes do wstrzymania operacji, unikając uderzenia w bazę SQL. Ogranicza to liczbę wywołań w cenie zaledwie kosztu 1 RCU i 1 WCU przy odrzuceniach.
4. Po sukcesie operacji następuje natychmiastowa transformacja w bazie do statusu COMPLETED, a zapisane wyjście powstrzyma wszelkie dalsze operacje tego konkretnego rekordu po wsze czasy. Wykorzystanie udoskonalenia ReturnValuesOnConditionCheckFailure udostępnionego niedawno w SDK Amazon DynamoDB, natychmiast pozyskuje wynik starego stanu w tym samym wywołaniu sieciowym, oszczędzając 50% czasu opóźnień.

Krok 5: Stabilizacja Relacyjnej Bazy (Amazon RDS Proxy & "Proxy Splitting")

Ponieważ system, wspierany przez tak potężną matrycę przesyłową, może w teorii wygenerować skalę przetwarzania do tysięcy instancji Lambda, konieczna jest racjonalizacja połączenia do relacyjnej bazy danych Amazon Aurora lub RDS PostgreSQL. Zastosowanie Amazon RDS Proxy stwarza natywną wieloużywalność (multiplexing) pojedynczego fizycznego gniazda. Jednak dogłębna analiza operacyjna demaskuje tu destruktywną blokadę - tzw. **Connection Pinning**. Dokumentacja RDS stwierdza kategorycznie, iż zapytanie SQL, którego sam czysty format tekstowy posiada rozmiar nadrzędny wobec 16 KB, automatycznie unieważnia wszystkie dobrodziejstwa współdzielonej puli. Proxy trwale "przypina" żądane połączenie do danej sesji ucinając jego wieloużywalność z uwagi na możliwość zmiany wewnętrznych struktur stanu bazy.

Przesłanie rozbudowanego i wzbogaconego Webhooka o skomplikowanej zawartości w pliku

JSONB bezproblemowo przekracza limity. A zatem produkcyjnie musi nastąpić implementacja wzorca nazwanego jako **Proxy Splitting** z poziomu kodu strukturalnego NestJS.

Podejście produkcyjne obejmuje inicjalizację minimum dwóch instancji logiki bazy:

// Konstrukcja dla Standardowych / Małych odczytów:

```
TypeOrmModule.forRootAsync({
  imports: [Con[span_33](start_span)[span_33](end_span)figModule],
  dataSourceFactory: async (options: DataSourceOptions) => {
    // Nawigacja przez domyślny, wysoce współdzielony punkt dostępowy RDS Proxy
    const AppDataSource = new DataSource(options);
    return await AppDataSource.initialize();
  },...
}),
```

// Konstrukcja dla Olbrzymich, wielkobajtowych zdarzeń (np. Raportowanie obciążeń Payloadów):

```
TypeOrmModule.forRootAsync({
  name: 'heavy-write-datasource',
  imports: [ConfigModule],
  dataSourceFactory: async (options: DataSourceOptions) => {
    // Nawigacja przez zdefiniowany wyizolowany punkt AWS (Proxy Splitting)
    // gwarantujący separację zasobów - omijając paraliż Connection Pinning:
    const SecondDataSource = new DataSource(options);
    return await SecondDataSource.initialize();
  },...
})
```

Moduły dedykowane (Repositories) przechowujące krytyczne ładunki wstrzykują tę wyizolowaną strukturę, zabezpieczając system jako całość przed atakami objętościowymi lub pęknięciami infrastrukturalnymi.

Powyższe opracowanie, łączące tytaniczną siłę technologii bezserwerowych (Pipes), sztucznej inteligencji, bezpieczeństwa na poziomie brzegowym i niepodważalnego paradygmatu idempotencji opartego na stanach maszyny optymalizuje architekturę do standardu gotowego na nadejście kolejnej dekady skalowania przetwarzania informacji w chmurze obliczeniowej.

Cytowane prace

1. AWS Lambda Cold Starts: The Case of a NestJS Mono-Lambda API - DEV Community, <https://dev.to/aws-builders/aws-lambda-cold-starts-the-case-of-a-nestjs-mono-lambda-api-4j42>
2. Improve NestJS cold starts on Lambda with RXJS | by Rudyard Murdough - Medium, https://medium.com/@rudyard_55741/improve-nestjs-cold-starts-on-lambda-with-rxjs-5dde21675e54
3. Serverless Love Story: NestJS & Lambda — Part I: Minimizing Cold Starts - Towards AWS, <https://towardsaws.com/serverless-love-story-nestjs-lambda-part-i-minimizing-cold-starts-4ba513e5ce02>
4. Serverless - FAQ | NestJS - A progressive Node.js framework, <https://docs.nestjs.com/faq/serverless>
5. Sending and receiving webhooks on AWS: Innovate with event ..., <https://aws.amazon.com/blogs/compute/sending-and-receiving-webhooks-on-aws-innovate-with-event-notifications/>
6. Input to an API Gateway Lambda authorizer - AWS Documentation,

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-lambda-authorizer-input.html> 7. Secure your API Gateway APIs with Lambda Authorizer - Jimmy Dahlqvist, <https://jimmysdavis.com/secure-api-gw-with-lambda-auth/> 8. Get request body from custom lambda authorizer | AWS re:Post, <https://repost.aws/questions/QU2A7CgJUCQ1ivbOrAWn7rHw/get-request-body-from-custom-lambda-authorizer> 9. Get request body from custom lambda authorizer - Stack Overflow, <https://stackoverflow.com/questions/77648642/get-request-body-from-custom-lambda-authorizer> 10. Raw Body | NestJS - A progressive Node.js framework, <https://docs.nestjs.com/faq/raw-body> 11. Access raw body of Stripe webhook in Nest.js - Stack Overflow, <https://stackoverflow.com/questions/54346465/access-raw-body-of-stripe-webhook-in-nest-js> 12. How to access the raw body of a Stripe webhook request in NestJS - Manuel Heidrich, <https://manuelheidrich.dev/blog/how-to-access-the-raw-body-of-a-stripe-webhook-request-in-nestjs/> 13. Security Enhancement: Add JWT Signature Verification in Downstream Lambda Functions #93 - GitHub, <https://github.com/aws-solutions/innovation-sandbox-on-aws/issues/93> 14. Understanding Webhooks and How to Connect Them with NestJS | by hamdi rebhi | Medium, <https://medium.com/@hamdi22rebhk/understanding-webhooks-and-how-to-connect-them-with-nestjs-a93cbc05ba48> 15. How to Build Claim Check Pattern - OneUptime, <https://oneuptime.com/blog/post/2026-01-30-claim-check-pattern/view> 16. Managing large Amazon SQS messages using Java and Amazon S3 - AWS Documentation, <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-s3-messages.html> 17. More room to build: serverless services now support payloads up to 1 MB - AWS, <https://aws.amazon.com/blogs/compute/more-room-to-build-serverless-services-now-support-payloads-up-to-1-mb/> 18. Using Amazon RDS Proxy with AWS Lambda | AWS Compute Blog, <https://aws.amazon.com/blogs/compute/using-amazon-rds-proxy-with-aws-lambda/> 19. RDS Proxy v. Connection Pooling in Lambda : r/aws - Reddit, https://www.reddit.com/r/aws/comments/1fi9734/rds_proxy_v_connection_pooling_in_lambda/ 20. Avoiding Connection Pinning in Lambda and RDS Proxy with ..., <https://dev.to/suzuki0430/avoiding-connection-pinning-in-lambda-and-rds-proxy-with-nestjs-and-proxy-splitting-2l44> 21. Highly Available Database Proxy – Amazon RDS Proxy - AWS, <https://aws.amazon.com/rds/proxy/> 22. Handling Lambda functions idempotency with AWS Lambda Powertools | AWS Compute Blog, <https://aws.amazon.com/blogs/compute/handling-lambda-functions-idempotency-with-aws-lambda-powertools/> 23. Idempotency in AWS: How to Prevent Duplicate Operations at Scale - Joud W. Awad, <https://joudwawad.medium.com/idempotency-in-aws-how-to-prevent-duplicate-operations-at-scale-b28ce9d69a57> 24. Idempotency in Distributed Systems: Design Patterns Beyond 'Retry Safely' | Alok, <https://aloknecessary.github.io/blogs/idempotency-distributed-systems/> 25. NestJS SQS Listener: A Practical & Type-Safe Way to Consume SQS - Medium, <https://medium.com/@ganesan.arunachalam91/nestjs-sqs-listener-a-practical-type-safe-way-to-consume-sqs-4b2dee861aef> 26. Integrating NestJS with Amazon SQS: A Comprehensive Guide - Medium, <https://medium.com/@horan.dev/integrating-nestjs-with-amazon-sqs-a-comprehensive-guide-5f7a2c004853> 27. Edge Computing 2026: Web Performance Architecture - Digital Applied, <https://www.digitalapplied.com/blog/edge-computing-2026-web-performance-architecture> 28. JavaScript runtime 1.0 features for CloudFront Functions - AWS Documentation,

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/functions-javascript-runtime-10.html> 29. Use async and await - Amazon CloudFront, <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/async-await-syntax.html> 30. Validate a simple token in a CloudFront Functions viewer request - AWS Documentation, https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/example_cloudfront_functions_kvs_jwt_verify_section.html 31. Integrate Amazon API Gateway with Amazon SQS to handle asynchronous REST APIs, <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/integrate-amazon-api-gateway-with-amazon-sqs-to-handle-asynchronous-rest-apis.html> 32. AWS Api Gateway connect to SQS with message attributes - Stack Overflow, <https://stackoverflow.com/questions/52877403/aws-api-gateway-connect-to-sqs-with-message-attributes> 33. Integrate a REST API with Amazon SQS and resolve errors - AWS re:Post, <https://repost.aws/knowledge-center/api-gateway-rest-api-sqs-errors> 34. Building event-driven pipelines with SQS and S3 - Redpanda, <https://www.redpanda.com/blog/building-event-driven-pipelines-sqs-s3> 35. Claim Check Pattern with AWS SQS, SAP PO and KaTe AWS Adapter - SAP Community, <https://community.sap.com/t5/technology-blog-posts-by-members/claim-check-pattern-with-aws-sqs-sap-po-and-kate-aws-adapter/ba-p/13454905> 36. Claim-Check Pattern with AWS Message Processing Framework for .NET and Aspire, <https://nikiforovall.blog/dotnet/aws/2024/05/27/aws-claim-check-dotnet.html> 37. Architecting Serverless Webhooks with AWS: Best Practices and Use Cases, <https://www.businesscompassllc.com/architecting-serverless-webhooks-with-aws-best-practices-and-use-cases/> 38. Guidance for Self-Healing Code on AWS - AWS Documentation, <https://docs.aws.amazon.com/solutions/self-healing-code-on-aws/> 39. Unbreakable DevOps Pipeline: Shift-Left, Shift-Right & Self-Healing - Dynatrace, <https://www.dynatrace.com/news/blog/unbreakable-devops-pipeline-shift-left-shift-right-self-healing/> 40. Building Self-Healing Infrastructure-as-Code with Dynatrace, AWS Lambda, and AWS Service Catalog | AWS Partner Network (APN) Blog, <https://aws.amazon.com/blogs/apn/building-self-healing-infrastructure-as-code-with-dynatrace-aws-lambda-and-aws-service-catalog/> 41. Implementing a Self-Healing Serverless CI/CD Pipeline with AWS Developer Tools., <https://dev.to/aws-builders/implementing-a-self-healing-serverless-cicd-pipeline-with-aws-developer-tools-611> 42. Accessing Amazon EventBridge Pipes through the Amazon SQS console, <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/eb-pipes-integration.html> 43. EventBridge Pricing Calculator - Event Buses, Pipes, Scheduler & More - CloudBurn, <https://cloudburn.io/tools/amazon-eventbridge-pricing-calculator> 44. Scale Kafka Consumers on Demand: A Practical Guide to Autoscaling in Real-Time Systems, <https://medium.com/@vinodbokare0588/kafka-consumer-scaling-the-eventbridge-pattern-that-saves-60-costs-fc63fc6f6056> 45. Amazon Simple Queue Service as a source in EventBridge Pipes, <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes-sqs.html> 46. AWS EventBridge Pipes Deep Dive | by Joud W. Awad - Medium, <https://joudwawad.medium.com/aws-eventbridge-pipes-deep-dive-3ea98c0507e2> 47. Amazon EventBridge Pipes targets - AWS Documentation, <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes-event-target.html> 48. Enriching and customizing notifications with Amazon EventBridge Pipes - AWS, <https://aws.amazon.com/blogs/compute/enriching-and-customizing-notifications-with-amazon-eventbridge-pipes/> 49. Examples of applying pessimistic and optimistic concurrency control with AWS DynamoDB. - GitHub, <https://github.com/filipsnastins/concurrency-control-with-dynamodb>

50. Idempotency - Powertools for AWS Lambda (TypeScript),
<https://docs.aws.amazon.com/powertools/typescript/1.18.0/utilities/idempotency/> 51.
Idempotency - Powertools for AWS Lambda (.NET),
<https://docs.aws.amazon.com/powertools/dotnet/utilities/idempotency/> 52. Handle conditional
write errors in high concurrency scenarios with Amazon DynamoDB,
<https://aws.amazon.com/blogs/database/handle-conditional-write-errors-in-high-concurrency-scenarios-with-amazon-dynamodb/> 53. ConditionExpression in DynamoDB update operation gives
unknown error - Stack Overflow,
<https://stackoverflow.com/questions/63646742/conditionexpression-in-dynamodb-update-operation-gives-unknown-error> 54. Prevent duplicate Lambda events | AWS re:Post,
<https://repost.aws/knowledge-center/lambda-function-idempotent>